DESARROLLADOR DE APLICACIONES

Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

0484 MP Bases de Datos
UD4 Tratamiento de datos

Índice

01. Introducción

02. Inserción de registros

2.1. Inserción de uno o varios registros: sentencia INSERT

2.2. Inserción de grandes cantidades de registros

03. Actualización de registros: sentencia UPDATE

04. Eliminación de registros: sentencia DELETE

05. Creación de bases de datos, tablas y vistas: sentencia CREATE

5.1. Creación de la base de datos: sentencia CREATE DATABASE

5.2. Creación de tablas: sentencia CREATE TABLE

5.3. Creación de vistas: sentencia CREATE VIEW

06. Eliminación de bases de datos, tablas y vistas: sentencia DROP

07. Modificación en la definición de las tablas: sentencia ALTER TABLE

08. Transacciones

09. Políticas de bloqueo

010. Bibliografía

01. Introducción

El lenguaje de manipulación de datos (DML, Data Manipulation Language) incluye las instrucciones que me van a permitir manipular el contenido de la base de datos, es decir me va a permitir la inserción de datos nuevos en las tablas o vistas, eliminación y actualización de esos datos o consulta de la información contenida en la base de datos. Por lo tanto, las sentencias DML del lenguaje SQL son:

- SELECT: permite la consulta de información almacenada en la base de datos.
- INSERT: permite insertar registros en alguna tabla de la base de datos.
- DELETE: borra registros de una tabla de la base de datos.
- UPDATE: modifica los registros de una tabla de la base de datos.

Por otro lado, el lenguaje de definición de datos (DDL, Data Definition Language) incluye las instrucciones que me van a permitir crear, eliminar y modificar la estructura de la base de datos (tablas, vistas, restricciones, etc.). Por lo tanto, las sentencias DDL del lenguaje SQL son:

- CREATE: permite crear bases de datos, tablas, vistas, etc.
- DROP: permite borrar tablas, vistas, restricciones o incluso la base de datos al completo.
- ALTER: modifica las tablas, vistas o restricciones de la propia base de datos.

La sentencia SELECT es, con diferencia, la sentencia más compleja de SQL, por lo que se ha dedicado todo un tema, el anterior, para su estudio. A lo largo de este tema, 4, se estudiará el resto de las sentencias del lenguaje DML (INSERT, UPDATE y DELETE), además de las sentencias del lenguaje DDL (CREATE, DROP y ALTER). Finalmente, aprenderemos a realizar copias de seguridad, así como la implementación de transacciones.

02. Inserción de registros

Podemos insertar nuevos registros (filas) en una tabla de una base de datos MySQL de distintas maneras. Si queremos insertar uno o varios registros (no gran cantidad), utilizaremos la sentencia INSERT, sin embargo, si queremos importar grandes cantidades de datos que están incluidos en un fichero CSV, XML o SQL,

usaremos respectivamente las sentencias LOAD DATA, LOAD XML o SOURCE. En este apartado veremos cómo utilizar cada una de estas sentencias.

2.1. Inserción de uno o varios registros: sentencia INSERT

La sentencia INSERT me va a permitir insertar una o varias filas en una tabla de mi base de datos. A continuación, veremos los distintos tipos de sintaxis que admite. La primera y más sencilla es:

```
INSERT INTO { nombre_tabla | nombre_vista } [ (lista_columnas) ]
VALUES  (lista_valores);
```

Donde:

- nombre_tabla o nombre_vista: corresponde al nombre de la tabla o la vista de la base de datos donde vamos a insertar los datos.
- lista_columnas: será la lista de columnas cuyos valores vamos a introducir. Puede haber columnas cuyo valor se desezconozca y por lo tanto no se introducirán.
- lista_valores: es el conjunto de valores (expresiones o literales) que se van a introducir en la tabla o vista de la base de datos.

Si se han especificado las columnas (lista_columnas), lista_valores se asociará de manera correlativa con los valores a las columnas indicadas. Como lista_columnas es opcional, en el caso de que no se especifiquen, lista_valores debe escribirse en el orden de las columnas en la definición de la tabla.

Todos los valores a insertar van entre comillas simples, salvo los valores numéricos que nunca se entrecomillan.

Ejemplo: Inserta en la tabla profesroes a César Mateos con DNI 65987452N, email cesar.materos@gmail.com y teléfono 652321452.

```
INSERT INTO profesor
VALUES
('65987452N', 'César', 'Mateos', 'cesar.materos@gmail.com', '652321452', NULL);
```

Si se desean insertar varios profesores simultáneamente utilizando una sola sentencia INSERT, se realizará como se muestra en el siguiente ejemplo.

Ejemplo: Inserta los profesores:

- Julia Pérez con DNI 12457843N, email julia.perez@ufv.es y número de teléfno 698745487.
- Blanca Sánchez, con DNI 54123654B, email b.sanchez@gmail.com,número de teléfono 632145854 y dirección Rúa Mayor, 15, Madrid.
- Pedro, con DNI 74854521L, teléfono 652145487 y email pedro@gmail.com.

```
INSERT INTO profesor
VALUES
('12457843N', 'Julia', 'Pérez', 'julia.perez@ufv.es', '698745487', NULL),
('54123654B', 'Blanca', 'Sánchez', 'b.sanchez@gmail.com', '632145854', 'Rúa
Mayor, 15, Madrid'),
('74854521L', 'Pedro', NULL, 'pedro@gmail.com', '652145487', NULL);
```

Si no disponogo de todos los datos, es decir de todas las columnas, del registro que voy a insertar, puedo abreviar la sentencia INSERT añadiendo la lista de columnas de los que sí dispongo datos. De esta manera evitaré tener que escribir los atributos en el mismo orden de la creación de la tabla, así como de rellenar con NULL los datos que desconozca.

Ejemplo: Inserta a la profesor Marcos con DNI 87546532X y email, marcos@gmail.com. Esto lo puedo realizar de dos maneras distintas:

```
INSERT INTO profesor (dni, nombre, email)
VALUES ('87546532X', 'Marcos', 'marcos@gmail.com');
```

O bien:

```
INSERT INTO profesor
VALUES ('87546532X', 'Marcos',  NULL, 'marcos@gmail.com', NULL,
NULL);
```

La sentencia INSERT también me va a permitir especificar los nombres y los valores de las columnas que voy a añadir explícitamente con SET. En este caso la sintaxis sería:

```
INSERT INTO { nombre_tabla | nombre_vista }
SET nombre_columna = valor_columna, …;
```

Ejemplo: Añade a la base de datos los datos de la alumna Carlota con dni 74857456P y email .

```
INSERT INTO alumno
SET nombre='Carlota', dni='74857456P', email=' carlota.madrid@gmail.com';
```

Por último, la sentencia INSERT permite obtener los datos que voy a insertar en una tabla de mi base de datos mediante una consulta a la propia base de datos. En este caso su sintaxis sería:

```
INSERT INTO { nombre_tabla | nombre_vista } [ (lista_columnas) ]
SELECT …
```

En este caso, las columnas devueltas por la SELECT se deben corresponder en número y orden con las columnas especificadas en lista_columnas. En caso de no especificarse lista_columnas, se deberá corresponder en número y orden con las columnas establecidas en la creación de la tabla.

Ejemplo: Se desea añadir a una tabla Suspensos_1Ev el dni de los alumnos con alguna asignatura suspensa junto con el número de suspensos que tiene en la primera evaluación. Es decir esta tabla Suspensos_1Ev, estará formada solamente por dos columnas, la primera será el DNI y la segunda Num_suspensos.

```
INSERT INTO Suspensos_1Ev
SELECT alumno, COUNT(*)
FROM alumno_asignatura
WHERE nota_1Ev<5
GROUP BY alumno;
```

2.2. Inserción de grandes cantidades de registros

Muchas veces resulta interesante insertar todos los datos incluidos en un fichero CSV, XML o SQL. La cantidad de registros que incluyen normalmente este tipo de ficheros es tan grande, que resulta impensable tener que insertar todos estos datos en las tablas de mi base de datos individualmente con sentencias INSERT.

Dependiendo del formato del fichero donde se encuentren los datos, CSV (o similar), XML o SQL, utilizaré una sentencia u otra. A continuación, estudiaremos cada uno de estos casos.

FICHERO DE DATOS CSV O SIMILAR: SENTENCIA LOAD DATA

La sentencia LOAD DATA me va a permitir insertar registros en una tabla de mi base de datos a gran velocidad, a partir de un fichero con cierto formato. Su sintaxis es:

```
LOAD DATA  [ LOCAL ]  INFILE  'fichero'
[ REPLACE | IGNORE ]
INTO TABLE nombre_tabla
[ {FIELDS | COLUMNS}
        [ TERMINATED BY 'string' ]
        [ [ OPTIONALLY ] ENCLOSED BY 'char' ]
        [ ESCAPED BY 'char' ] ]
[ LINES
        [STARTING BY 'string' ]
        [ TERMINATED BY 'string' ] ]
[IGNORE número LINES]
```

Donde:

- LOCAL: indica que el fichero especificado sea leído desde el cliente que realiza la conexión. En caso de no estar presente, el servidor entiende que debe leer el fichero en el propio servidor.
- INFILE 'fichero': indica el fichero que contiene los datos, pudiendo espedificarse con una ruta absoluta o relativa.
- REPLACE | IGNORE: permite que en caso de existir el valor de una clave primaria, el registro que se desea añadir con esa clave se actualice (REPLACE) o se omita continuando con el resto de inserciones (IGNORE).

- { FIELDS | COLUMNS } TERMINATED BY: sirve para delimitar el final de cada campo de datos, es decir, delimita cada atributo dentro de una tupla.
- { FIELDS | COLUMNS } OPTIONALLY ENCLOSED BY: indica que los campos pueden estar delimitados por un carácter, como por ejemplo comillas dobles.
- { FIELDS | COLUMNS } ESCAPED BY: indica el carácter que se usará para “escapar” los caracteres de manera que no se confunda con los usados para delimitar campos, indicar separador de campos y el carácter de fin de línea.
- LINES STARTING BY: indica que las líneas del fichero que contiene los datos comienzan por cierta cadena.
- LINES TERMINATED BY: indica la cadena que hay al final de cada línea en el fichero. Sirve para delimitar cada registro de datos, es decir, delimita cada tupla.
- IGNORE número LINES: sirve para especificar el número de líneas a omitir al principio del fichero. Sirve para cuando hay cabeceras o datos que no corresponden con campos de las tablas.

Los valores por defecto si omitimos la cláusula FIELDS y LINES son:

- FIELDS TERMINATED BY '\t' ENCLOSED BY '' ESCAPED BY '\\'
- LINES TERMINATED BY '\n' STARTING BY ''

Donde '\t' es el carácter tabulador y '\n' es un salto de línea.

Ejemplo: Disponemos de un fichero con datos de alumnos llamado "alumnos1.txt" y situado en la unidad C del equipo. En él, cada fila o registro está separada por un salto de línea y cada campo por un punto y coma. Indica el comando necesario para cargar los datos en la tabla de alumnos omitiendo los errores en la repetición de clave, así como las dos primeras líneas que indican información genérica sobre el contenido del archivo.
Este fichero alumnos1.txt lo puedes encontrar en la carpeta Recursos de este tema.

```
LOAD DATA LOCAL INFILE 'C:\\alumnos1.txt'
IGNORE INTO TABLE alumno
FIELDS TERMINATED BY ';'
LINES TERMINATED BY '\n'
IGNORE 2 LINES;
```

Ejemplo: Si los datos de los alumnos que se desean insertar en la base de datos se encuentran en un fichero "alumnos2.txt" situado también en la unidad C del equipo, pero en el que cada fila o registro está en una línea nueva y cada campo de ese registro se separa del siguiente por un carácter tabulador. ¿cuál sería el comando necesario para cargar los datos en la tabla de alumnos omitiendo los errores en la repetición de clave, así como las dos primeras líneas que al igual que antes indican información genérica sobre el contenido del archivo?
Este fichero alumnos2.txt también lo puedes encontrar en la carpeta Recursos de este tema.

```
LOAD DATA LOCAL INFILE 'C:\\alumnos2.txt'
IGNORE INTO TABLE alumno
IGNORE 2 LINES;
```

FICHERO DE DATOS XML: SENTENCIA LOAD XML

La sentencia LOAD XML me va a permitir insertar registros en una tabla de mi base de datos a gran velocidad, a partir de un fichero XML.

En un fichero XML, cada uno de los registros está codificado en el lenguaje de etiquetas XML de una de las dos maneras siguientes:

- Cada registro está identificado con la etiqueta "<row>". Cada campo de ese registro está delimitado con la etiqueta "<field>", donde el valor del atributo "name" es el nombre de la columna.

```xml
<?xml version="1.0"?>

<resultset statement="SELECT * FROM centroEstudios.profesor
" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <row>
    <field name="dni">12457843N</field>
    <field name="nombre">Julia</field>
    <field name="apellido">P,rez</field>
    <field name="email">julia.perez@ufv.es</field>
    <field name="telefono">698745487</field>
    <field name="direccion"></field>
  </row>

  <row>
    <field name="dni">45124785A</field>
    <field name="nombre">Jorge</field>
    <field name="apellido">Alonso</field>
    <field name="email">jorge.alonso@ufv.es</field>
    <field name="telefono">965875487</field>
    <field name="direccion">Calle Altamira, 25, Madrid</field>
  </row>

</resultset>
```

- Cada registro está identificado con la etiqueta "<row>". Las columnas aparecen como etiquetas y sus valores como contenido de las mismas.

```
<?xml version="1.0"?>

<resultset statement="SELECT * FROM centroEstudios.profesor
" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <row>
    <dni>12457843N</dni>
    <nombre>Julia</nombre>
    <apellido>P,rez</apellido>
    <email>julia.perez@ufv.es</email>
    <telefono>698745487</telefono>
    <direccion></direccion>
  </row>

  <row>
    <dni>45124785A</dni>
    <nombre>Jorge</nombre>
    <apellido>Alonso</apellido>
    <email>jorge.alonso@ufv.es</email>
    <telefono>965875487</telefono>
    <direccion>Calle Altamira, 25, Madrid</direccion>
  </row>

</resultset>
```

Dicho esto, la sintaxis del comando LOAD XML para insertar grandes cantidades de datos en nuestra base de datos a partir de ficheros en este formato XML es:

LOAD XML [LOCAL] INFILE 'fichero'
[REPLACE | IGNORE]
INTO TABLE [nombre_baseDatos.]nombre_tabla
[ROWS IDENTIFIED BY '<etiqueta>']
[IGNORE número [LINES | ROWS]]

Donde:

- ROWS IDENTIFED BY <etiqueta>: indica la etiqueta que iniciará cada fila en la tabla.

Ejemplo: Inserta los datos del fichero alumnos.xml, situado en la unidad C del equipo, en la base de datos Centro de Estudios. Este fichero (que puedes encontrar en la carpeta recursos de este tema) tiene el siguiente formato:

```
<row>
    <dni>74857485F</dni>
    <nombre>Mateo</nombre>
    <apellido>Ros</apellido>
    <email>mateo.ros@gmail.com</email>
    <telefono>654987656</telefono>
    <direccion></direccion>
    <fecha_nacimiento></fecha_nacimiento>
</row>

<row>
    <dni>12457845L</dni>
    <nombre>Marta</nombre>
    <apellido>Sanz</apellido>
    <email>m.sanz@gmail.com</email>
    <telefono></telefono>
    <direccion></direccion>
    <fecha_nacimiento></fecha_nacimiento>
</row>

<row>
    <dni>32659874X</dni>
    <nombre>Juan</nombre>
    <apellido>Alonso</apellido>
    <email>juanito@gmail.com</email>
    <telefono></telefono>
    <direccion></direccion>
    <fecha_nacimiento>1995-05-06</fecha_nacimiento>
</row>
```

El comando necesario para realizar esta inserción es:

```
LOAD XML LOCAL INFILE  'C:\\alumnos.xml'
REPLACE
INTO TABLE centroestudios.alumno
ROWS IDENTIFIED BY '<row>';
```

MySQL también permite exportar datos en formato XML, de tal manera que podemos realizar copias de seguridad de las distintas tablas y almacenar dichas copias en un fichero XML.

Esta exportación de datos en formato XML se debe realizar necesariamente en la consola de comandos. Para ello, en Windows, habrá que situarse, en la carpeta

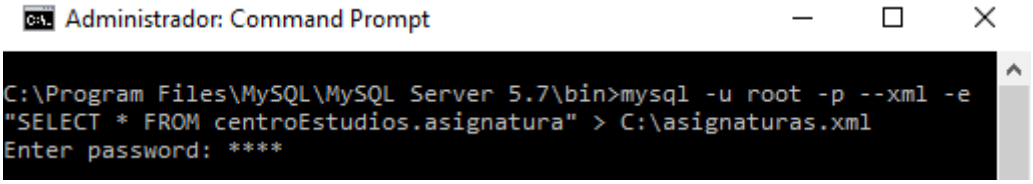
bin donde se instaló MySQL, (normalmente C:\Program Files\MySQL\MySQL Server 5.7\bin) y ejecutar el siguiente comando:

mysql -u usuario -p --xml -e "SELECT * FROM baseDatos.tabla" > fichero.xml

Donde:

- -u usuario indica que me conectaré con el usuario 'usuario', y la opción -p indica que me solicite el password.
- --xml -e "SELECT * FROM baseDatos.tabla": indica que se desea obtener el resultado de esa consulta en formato XML.
- fichero.xml: indica redireccionamiento de la salida estándar al fichero indicado, es decir, el resultado de la consulta no la voy a mostrar por pantalla, sino que lo voy a almacenar en fichero.xml.

Ejemplo: Realiza una copia de seguridad del contenido de la tabla asignaturas en formato XML.



FICHERO DE DATOS SQL: SENTENCIA SOURCE

Los registros que quiero insertar en alguna tabla de mi base de datos, pueden venir almacenados en ficheros SQL. Este tipo de ficheros están formados por cualquier comando SQL (INSERT, CREATE, DROP, DELECTE, ALTER, etc.…), por lo tanto, los nuevos registros que añadiré a mi base de datos vendrán codificados en forma de sentencias INSERT como las vistas anteriormente en este tema.

Si elaboro a mano este fichero SQL, es muy importante seleccionar la base de datos en la que voy a insertar los datos antes de realizar cualquier INSERT. Esto lo haré mediante el comando USE.

El archivo centroestudios.sql utilizado en el tema anterior para crear nuestra base de datos Centro de Estudios, no es más que un archivo SQL con estas

características. Como se vio en ese tema las instrucciones de este archivo SQL se pueden a través de la opción "Abrir Script SQL" del menú "Archivo" de Workbench.

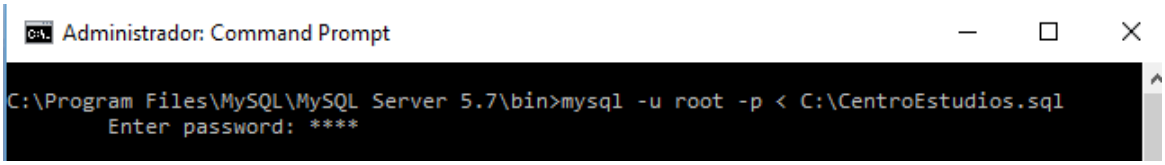
Otra forma de ejecutar los comandos SQL contenidos en un fichero es redireccionando los comandos del fichero al cliente MySQL usando las credenciales correspondientes. Para ello, en Windows, habrá que situarse, en la carpeta bin donde se instaló MySQL y ejecutar el siguiente comando:
mysql -u usuario -p < fichero.sql

Donde:

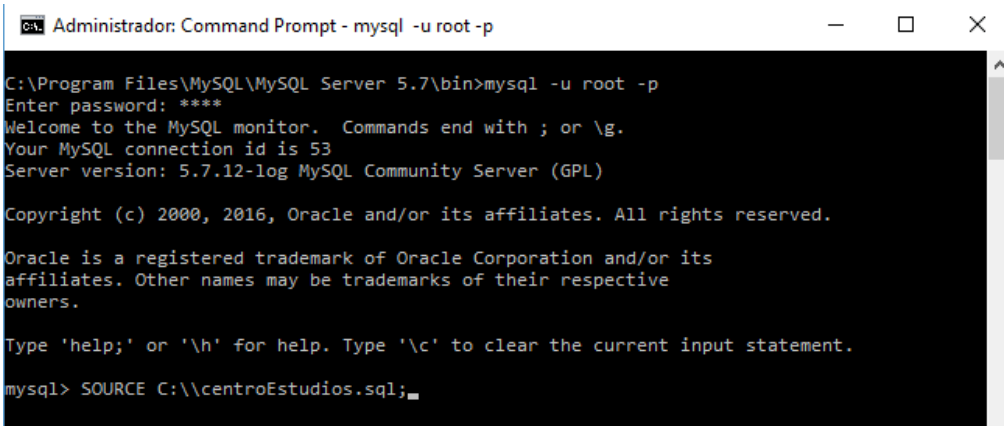
- -u usuario indica que me conectaré con el usuario 'usuario', y la opción -p indica que me solicite el password.
- < fichero.sql: indica redireccionamiento de la entrada estándar, es decir, los comandos que voy a ejecutar no los voy a escribir por teclado, sino que se encuentran almacenados en fichero.sql.

Si estamos conectados como clientes a la consola de MySQL, podemos hacer lo mismo con el comando SOURCE. Ojo este comando no se puede utilizar en Workbench, sólo en la consola de MySQL Su sintaxis es:
mysql> SOURCE fichero.sql;

Ejemplo: Escribe el comando necesario para ejecutar las sentencias incluidas en el fichero centroEstudios.sql



O bien:



De igual modo que MySQL permite exportar datos en formato XML, permite también la exportación de datos en formato SQL. Por lo tanto podemos realizar copias de seguridad de las distintas tablas y bases de datos y almacenar dichas copias en un fichero SQL.

mysqldump es el comando que me va a permitir realizar esta exportación de datos en formato SQL. Este comando se debe utilizar necesariamente en la consola de comandos, en Windows, concretamente habrá que situarse, en la carpeta bin donde se instaló MySQL, (normalmente C:\Program Files\MySQL\MySQL Server 5.7\bin). Es en esta posición donde ejecutaré el comando. Sus tres sintaxis básicas son:

mysqldump [opciones] nombre_baseDatos [tablas]

mysqldump [opciones] --DATABASES baseDatos1 [baseDatos2 baseDatos3 …]

mysqldump [opciones] --ALL-DATABASES

Admite muchas opciones que puedo consultar en la ayuda de MySQL, de las que destacaremos:

- -u usuario: indica que me conectaré con el usuario 'usuario'.
- -p: indica que me solicite el password.
- -t: provoca el volcado de todos los datos de las tablas seleccionadas en forma de órdenes INSERT, pero no incorporará la creación de las tablas (si no se indica esta opción, se incorporará siempre la creación de las tablas).

Ejemplo: Realiza una copia de seguridad del contenido de la tabla asignaturas en formato SQL.

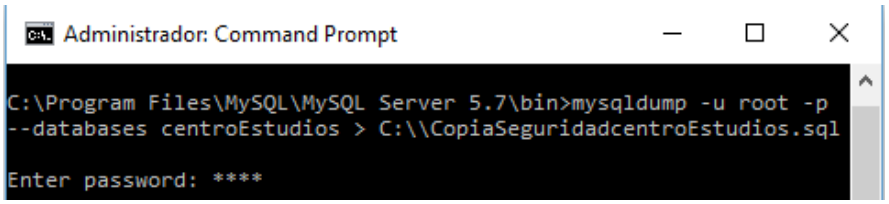
```
Administrador: Command Prompt

C:\Program Files\MySQL\MySQL Server 5.7\bin>mysqldump -u root -p centroEstudios Asignatura
Enter password: ****
-- MySQL dump 10.13  Distrib 5.7.12, for Win64 (x86_64)
--
-- Host: localhost    Database: centroEstudios
-- ------------------------------------------------------
-- Server version       5.7.12-log

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

--
-- Table structure for table `asignatura`
--

DROP TABLE IF EXISTS `asignatura`;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!40101 SET character_set_client = utf8 */;
CREATE TABLE `asignatura` (
  `codigo` varchar(11) NOT NULL,
  `nombre` varchar(45) NOT NULL,
  `descripcion` varchar(90) DEFAULT NULL
```

Como se observa en esta imagen este comando mostrará por pantalla todos los comandos SQL, pero no los tengo almacenados en un archivo para que el día que sea necesario, restaurar su contenido.

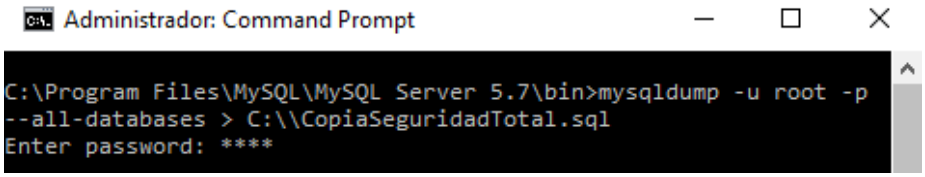
Para volcar todos estos datos en un fichero, y de esta manera crear un archivo donde se almancene la copia de seguridad, en este caso, de la tabla asignaturas, deberé redireccionar la salida estandar (pantalla) a un fichero con el carácter '>' tal y como hemos visto anteriormente en el tema.

```
Administrador: Command Prompt

C:\Program Files\MySQL\MySQL Server 5.7\bin>mysqldump -u root -p
centroEstudios Asignatura > C:\\asingatura.sql
Enter password: ****

C:\Program Files\MySQL\MySQL Server 5.7\bin>
```

Ejemplo: Crea un archivo SQL con la copia de seguridad de la base de datos Centro de Estudios.



Ejemplo: Crea un archivo SQL con la copia de seguridad de todas las bases de datos instaladas.



03. Actualización de registros: sentencia UPDATE

La sentencia UPDATE me va a permitir modificar el contenido de cualquier columna y de cualquier fila de las tablas de mi base de datos. Su sintaxis es:

UPDATE nombre_tabla
SET nombre_col1= { expr1 | DEFAULT } [, nombre_col2= { expr2 | DEFAULT } …]
[WHERE condición]
[ORDER BY columna_ordenación];

Ojo, si se omite la cláusula WHERE se modificarán todas las filas de la tabla.

Ejemplo: Aumenta 10 horas a cada una de las asignaturas del ciclo formativo de 'DAM'.

```
UPDATE asignatura
SET horas = horas+10 WHERE ciclo='DAM';
```

Ejemplo: Aumenta 10 horas a cada una de las asignaturas.

```
UPDATE asignatura
SET horas = horas+10;
```

Ejemplo: El apellido de la alumna Carlota, con DNI '74857456P' es Gonzáles. Añade esta información a la base de datos.

```
UPDATE alumno
SET apellido='González'
WHERE DNI= '74857456P';
```

04. Eliminación de registros: sentencia DELETE

La sentencia DELETE me va a permitir eliminar cualquier fila de las tablas de mi base de datos. Su sintaxis es:

```
DELETE FROM nombre_tabla
[WHERE condición]
[ORDER BY columna_ordenación];
```

Al igual que en la sentencia UPDATE, si se omite la cláusula WHERE se modificarán todas las filas de la tabla.

Ejemplo: La alumna Carlota González, con DNI '74857456P' se ha dado de baja en el centro, elimina por tanto su información de la base de datos.

```
DELETE FROM alumno
WHERE DNI= '74857456P';
```

Ejemplo: Borra todo el contenido de la tabla alumno.

```
DELETE FROM alumno;
```

05. Creación de bases de datos, tablas y vistas: sentencia CREATE

Como se vio en el tema 2, el modelo relacional me va a indicar qué tablas crear, qué atributos forman esas tablas, así como la relación entre las distintas tablas. En este apartado aprenderemos a implementar el modelo relacional diseñado en lenguaje SQL. Para ello, estudiaremos en primer lugar cómo crear una base de datos, en segundo lugar, cómo crear las tablas de esa base de datos, y finalmente, cómo crear vistas en una base de datos.

5.1. Creación de la base de datos: sentencia CREATE DATABASE

La sentencia CREATE DATABASE, como su propio nombre indica, me va a permitir crear una base de datos vacía, sin ninguna tabla. Su sintaxis es:

CREATE { DATABASE | SCHEMA } [IF NOT EXISTS] nombre_baseDatos;

Ejemplo: Crea una base de datos llamada tienda.

```
CREATE DATABASE tienda;
```

Una vez creada la base de datos, para poder utilizarla, lo primero que hay que hacer es indicar que las operaciones que se realicen a continuación, se ejecuten sobre esta base de datos. Esto lo indico con el comando

```
USE nombre_baseDatos;
```

En nuestro ejemplo:

```
USE tienda;
```

5.2. Creación de tablas: sentencia CREATE TABLE

Para crear las tablas de una base de datos, utilizo el comando CREATE TABLE. Su sintaxis es:

```
CREATE TABLE [IF NOT EXISTS] nombre_tabla
[ (create_definition, ...  ) ]
[ table_options ];
```

Donde:

- create_definition:
  column_definition
  [CONSTRAINT [symbol]] PRIMARY KEY (col_name,...)]
  [KEY [name] (col_name,...)]
  [CONSTRAINT [symbol]] UNIQUE [name] (col_name,...)]
  [CONSTRAINT [symbol]] FOREIGN KEY [name] (col_name,...)
  [reference_definition]]
  [CHECK (expr)]

- column_definition:
  col_name type
  [NOT NULL | NULL]
  [DEFAULT default_value]
  [AUTO_INCREMENT]
  [UNIQUE [KEY] | [PRIMARY] KEY]

- reference_definition:
  REFERENCES tbl_name [(index_col_name,...)]
  [ON DELETE { RESTRICT | CASCADE | SET NULL | NO ACTION }]
  [ON UPDATE { RESTRICT | CASCADE | SET NULL | NO ACTION }]

- type:
  INTEGER[(length)] [UNSIGNED] [ZEROFILL]
  BIGINT[(length)] [UNSIGNED] [ZEROFILL]
  DOUBLE[(length,decimals)] [UNSIGNED] [ZEROFILL]
  FLOAT[(length,decimals)] [UNSIGNED] [ZEROFILL]
  DECIMAL(length,decimals) [UNSIGNED] [ZEROFILL]
  DATE
  TIME

DATETIME
CHAR(length) [BINARY | ASCII | UNICODE]
VARCHAR(length) [BINARY]
TEXT [BINARY]

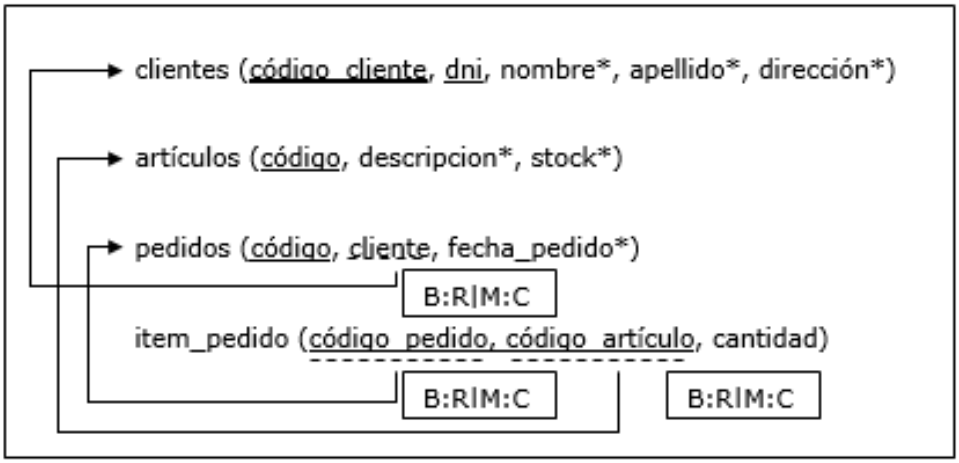
Puedes consultar más tipos de datos y comprobar cuándo se utiliza uno u otro en la ayuda de MySQL:

- table_options:
  {ENGINE|TYPE} = engine_name
  [DEFAULT] CHARACTER SET charset_name [COLLATE collation_name]
  COMMENT = 'string'
  MAX_ROWS = value
  MIN_ROWS = value

- engine_name:
  InnoDB
  MyISAM

En la ayuda de MySQL, puedes consultar para qué se utilizan cada una de las opciones especificadas: https://dev.mysql.com/doc/refman/8.0/en/create-table.html
La sintaxis de esta sentencia CREATE TABLE se entenderá mucho mejor con un ejemplo.

Ejemplo: Crear en MySQL las tablas de la base de datos tienda tal y como se muestran en el siguiente modelo relacional.



- Crear la tabla "clientes":

```
CREATE TABLE IF NOT EXISTS tienda.clientes (
codigo_cliente int(11) NOT NULL AUTO_INCREMENT UNIQUE KEY,
dni int(11) NOT NULL PRIMARY KEY,
nombre varchar(20),
apellido varchar(20),
direccion varchar(50) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

- Insertar datos en la tabla "clientes":

```
INSERT INTO tienda.clientes
VALUES (0, '77777777', 'Pedro', 'Martínez', 'Calle  Alcalá, 103'),
                (0, '88888888', 'Juan', 'Alonso', NULL),
                (0, '99999999', 'Alicia', NULL, NULL);
```

- Crear la tabla "artículos":

```
CREATE TABLE IF NOT EXISTS tienda.articulos (
codigo int(11) NOT NULL,
descripcion varchar(50),
stock int(11),
PRIMARY KEY (codigo)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

- Insertar datos en la tabla "artículos":

```
INSERT INTO tienda.articulos
VALUES (1, 'lapicero', 120),
                (2, 'boli bic negro', 350),
                (3, 'boli bic azul', NULL);
```

- Crear la tabla "pedidos":

```
CREATE TABLE IF NOT EXISTS tienda.pedidos (
codigo int(11) NOT NULL PRIMARY KEY,
cliente int(11) NOT NULL,
fecha_pedido timestamp NOT NULL
DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (cliente) REFERENCES tienda.clientes(dni)
        ON DELETE NO ACTION ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

- Mostrar las características de la tabla "pedidos":

```
SHOW CREATE TABLE tienda.pedidos;
```

- Insertar datos en la tabla "pedidos":

```
INSERT INTO tienda.pedidos
VALUES (1, '77777777', NULL),
                (2, '88888888', NULL);
```

- Borrar el cliente con dni "77777777". ¿Puedo borrar este cliente?, ¿por qué?:

```
DELETE FROM tienda.clientes
WHERE dni='77777777';
```

- Actualizar el cliente con dni "77777777" al dni “22222222”. ¿Qué ocurre con los pedidos asociados al cliente con dni "77777777"?:

```
UPDATE tienda.clientes
SET dni='22222222'
WHERE dni='77777777';
```

- Crear la tabla "item_pedido":

```
CREATE TABLE IF NOT EXISTS tienda.item_pedido (
codigo_pedido int(11) NOT NULL,
codigo_articulo int(11) NOT NULL,
cantidad int(11) NOT NULL DEFAULT 1,
PRIMARY KEY (codigo_pedido, codigo_articulo),
FOREIGN KEY (codigo_pedido)
REFERENCES tienda.pedidos(codigo)
        ON DELETE NO ACTION ON UPDATE CASCADE,
FOREIGN KEY (codigo_articulo)
REFERENCES tienda.articulos(codigo)
        ON DELETE NO ACTION ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

- Mostrar las características de la tabla "item_pedido":

```
SHOW CREATE TABLE tienda.item_pedido;
```

- Insertar datos en la tabla "item_pedido":

```
INSERT INTO tienda.item_pedido
VALUES (1, 1, DEFAULT), (1, 2, 2),
                (1, 3, 2), (2, 1, 1),
                (2, 2, 3), (2, 3, 3);
```

- Borrar el artículo cuyo código es "3" ¿Puedo borrar este artículo?, ¿por qué?:

```
DELETE FROM tienda.articulos
WHERE codigo=3;
```

- Actualizar el artículo con código "3" a código "4" ¿Qué ocurre con los item_pedido que contienen ese artículo con código "3".

```
UPDATE tienda.articulos
SET codigo=4 WHERE codigo=3;
```

5.3. Creación de vistas: sentencia CREATE VIEW

Una vista de una base de datos, también llamada tabla virtual, es el resultado de una consulta sobre una o varias tablas, es decir, su contenido será el resultado de la ejecución de una sentencia SELECT. Para la creación de vistas se utiliza el comando CREATE VIEW cuya sintaxis es:

```
CREATE VIEW  nombre_vista [ ( columnas ) ]
AS sentencia_select
[ WITH CHECK OPTION ];
```

Por defecto, los nombres de las columnas devueltos por la sentencia SELECT se usan también como nombres de las columnas de la vista. Si se desea dar explícitamente un nombre a las columnas de la vista éstas deben especificarse en "columnas", de tal manera que la cantidad de nombres indicados en "columnas" debe ser igual a la cantidad de columnas devueltas por la sentencia SELECT.

La vista puede ser siempre utilizada como tabla en posteriores consultas. Sin embargo, para esta vista puede ser utilizada para actualizar datos sólo si cumple determinadas restricciones:

- La SELECT sólo recupera filas de una única tabla, es decir hay una única tabla base.
- Todas las columnas seleccionadas se corresponde con columnas de la tabla base, por lo tanto no se utilizan funciones.
- La SELECT no incluye las cláusulas DISTINCT, GROUP BY, ni HAVING.
- La inserción sólo será posible si todas las columnas con NOT NULL y sin valor por defecto de la tabla original se recuperan en la SELECT.

Si se incluye la cláusula WITH CHECK OPTION no se permitirá que mediante un INSERT o UPDATE se cree una fila que no cumpla las condiciones de la vista.

Ejemplo: Crea una vista con los datos de los profesores que impartan alguna asignatura en el ciclo formativo de DAM.

```
CREATE VIEW profesores_DAM
AS SELECT distinct p.*
        FROM asignatura a, profesor p
        WHERE a.profesor=p.dni AND a.ciclo='DAM';
```

Para mostrar el nombre y apellido de los profesores que imparten algun asignatura en el ciclo formativo de DAM ejecutaré el siguiente comando:

```
SELECT nombre, apellido
FROM profesores_DAM;
```

No puedo realizar actualizaciones sobre esta vista "profesores_DAM ya que la sentencia SELECT incluye la cláusula DISTINCT.

Ejemplo: Crea una vista con los datos de las asignaturas que se imparten en el ciclo formativo de DAM.

```
CREATE VIEW asignaturas_DAM
AS SELECT a.*
        FROM asignatura a
        WHERE a.ciclo='DAM'
WITH CHECK OPTION;
```

Para mostrar el nombre y apellido de los profesores que imparten algun asignatura en el ciclo formativo de DAM ejecutaré el siguiente comando:

```
SELECT nombre, ciclo
FROM  asignaturas_DAM;
```

En este caso sí puedo realizar modificaciones sobre la vista "asignaturas_DAM", siempre y cuando las actualizaciones cumplan la condición de que el ciclo de la asignatura es DAM. Por ejemplo, puedo aumentar en 5 el número de horas de la asignatura Base de datos cuyo código es DAM_BBDD:

```
UPDATE asignaturas_DAM
SET horas=horas+5
WHERE codigo='DAM_BBDD';
```

Pero no puedo modificar el ciclo de esa misma asignatura, ya que no cumpliría la condición de que ciclo='DAM'.

```
UPDATE asignaturas_DAM
SET ciclo='AYF'
WHERE codigo='DAM_BBDD';
```

Si no hubiera creado la vista con la opción "WITH CHECK OPTION", sí que me hubiera dejado realizar esta actualización, aunque por supuesto, esta asignatura DAM_BBDD ya no se formaría parte de la vista asignaturas_DAM.

Es importante recordar que la vista solamente almacena la definición de la consulta, no almacena todos los datos de la tabla base de nuevo. Por tanto, cuando realizo una actualización sobre una vista, lo que realmente estoy modificando es la tabla base original.

06. Eliminación de bases de datos, tablas y vistas: sentencia DROP

Para borrar las bases de datos, tablas o vistas se utiliza la sentencia DROP. Su sintaxis es:

DROP { DATABASE | TABLE | VIEW } nombre_objeto;

Ejemplo: Borra las vistas creadas en el apartado anterior: asignaturas_DAM y profesores_DAM de la base de datos centro de estudios.

```
USE centroEstudios;
DROP VIEW asignaturas_DAM, profesores_DAM;
```

O lo que es lo mismo:

```
DROP VIEW centroEstudios.asignaturas_DAM,
          centroEstudios.profesores_DAM;
```

Ejemplo: Borra la tabla item_pedido de la base de datos tienda.

```
USE tienda;
DROP TABLE item_pedido;
```

O lo que es lo mismo:

```
DROP TABLE tienda.item_pedido
```

Ejemplo: Borra la base de datos tienda.

```
DROP DATABASE tienda;
```

Ojo, es muy distinto borrar una tabla, para lo cual utilizo la sentencia DROP, a borrar el contenido de una tabla, para lo que utilizo la sentencia DELETE.

07. Modificación en la definición de las tablas: sentencia ALTER TABLE

La sentencia ALTER TABLE permite añadir y eliminar columnas y restricciones de integridad a tablas ya creadas. Su sintaxis es:

```
ALTER TABLE nombre_tabla
alter_specification [, alter_specification ...];
```

Donde:

- alter_specification:
  ADD [COLUMN] especificación_columna
  | ADD [CONSTRAINT [symbol]] PRIMARY KEY (index_col_name,...)
  | ADD [CONSTRAINT [symbol]] UNIQUE [name] (col_name,...)
  | ADD [CONSTRAINT [symbol]] FOREIGN KEY [name] (col_name,...)
  [reference_definition]
  | CHANGE [COLUMN] old_col_name especificación_columna
  | DROP [COLUMN] col_name

```
| DROP PRIMARY KEY
| DROP FOREIGN KEY fk_symbol
```

Ejemplo: En la base de datos centro de estudios, añada un atributo NumSuspensos en la tabla alumno. Como su propio nombre indica, este campo contiene el número total de suspensos que tiene cada alumno. Su valor por defecto es 0.

```
ALTER TABLE alumno
ADD NumSuspensos INTEGER(2) DEFAULT 0;
```

Ejemplo: Borra el atributo NumSuspensos de la tabla alumno creado en el ejercicio anterior.

```
ALTER TABLE alumno
DROP NumSuspensos;
```

Ejemplo: Con una sola sentencia ALTER TABLE, añada en la base de datos centro de estudios un atributo NumSuspensos en la tabla alumno con valor por defecto es 0 y elimine de esa misma tabla la columna fecha_nacimiento.

```
ALTER TABLE alumno
ADD NumSuspensos INTEGER(2) DEFAULT 0,
DROP COLUMN fecha_nacimiento;
```

Ejemplo: Elimina la clave ajena que hay sobre el campo profesor de la tabla asignatura en la base de datos centro de estudios.
Para poder eliminar una clave ajena necesito el dato "fk_symbol", este dato lo puedo obtener viendo cómo está creada la tabla. Para ello ejecuto el comando:

```
SHOW CREATE TABLE asignatura;
```

Cuyo resultado se muestra en la imagen siguiente, y donde se observar que el "fk_symbol" para esta clave ajena es "asignatura_ibfk_1".

Edit Data for Create Table (VARCHAR)

Binary Text

```
1   CREATE TABLE `asignatura` (
2     `codigo` varchar(11) NOT NULL,
3     `nombre` varchar(45) NOT NULL,
4     `descripcion` varchar(90) DEFAULT NULL,
5     `horas` int(4) DEFAULT NULL,
6     `ciclo` varchar(15) NOT NULL,
7     `curso` int(2) DEFAULT NULL,
8     `profesor` varchar(11) DEFAULT NULL,
9     PRIMARY KEY (`codigo`),
10    KEY `profesor` (`profesor`),
11    CONSTRAINT `asignatura_ibfk_1` FOREIGN KEY (`profesor`) REFERENCES
    `profesor` (`dni`) ON DELETE SET NULL ON UPDATE CASCADE
12  ) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

Este dato se puede especificar en el momento de crear la clave ajena (bien cuando cree la tabla o bien si añado la clave ajena posteriormente una vez creada la tabla), pero es un dato opcional, sino se especifica, MySQL le da un valor automáticamente. Es recomendable dejar que MySQL asigne estos valores ya que tienen que ser nombres únicos en la base de datos.

```
ALTER TABLE asignatura
DROP FOREIGN KEY asignatura_ibfk_1;
```

Si ejecuto ahora el comando SHOW CREATE TABLE asignatura, se observa que ya no existe la clave ajena.

Edit Data for Create Table (VARCHAR)

Binary Text

```
1   CREATE TABLE `asignatura` (
2     `codigo` varchar(11) NOT NULL,
3     `nombre` varchar(45) NOT NULL,
4     `descripcion` varchar(90) DEFAULT NULL,
5     `horas` int(4) DEFAULT NULL,
6     `ciclo` varchar(15) NOT NULL,
7     `curso` int(2) DEFAULT NULL,
8     `profesor` varchar(11) DEFAULT NULL,
9     PRIMARY KEY (`codigo`),
10    KEY `profesor` (`profesor`)
11  ) ENGINE=InnoDB DEFAULT CHARSET=latin1
```

Ejemplo: Añada una clave ajena sobre el campo profesor de la tabla asignatura en la base de datos centro de estudios. Esta clave debe referenciar a la tabla profesor.

```
ALTER TABLE asignatura
ADD FOREIGN KEY (profesor) REFERENCES profesor(dni)
        ON DELETE SET NULL ON UPDATE CASCADE;
```

Ojo, es muy distinto modifica la definición de una tabla, para lo cual utilizo la sentencia ALTER TABLE, a modificar el contenido de una tabla, para lo que utilizo la sentencia UPDATE.

08. Transacciones

Una transacción es un conjunto de comandos SQL que se ejecutan de manera indivisible, es decir o se ejecutan todos o no se ejecuta ninguno. Las transacciones deben garantizar todas las propiedades ACID:

- Atomicidad: asegurar que se ejecutan todos los comandos o no se ejecuta ninguno, no puede quedar a medias.
- Consistencia: asegura que sólo empieza lo que se puede acabar.
- Aislamiento: asegura que ninguna operación afecta a otras pudiendo causar errores.
- Durabilidad: asegura que, una vez realizada una operación, ésta no podrá cambiar y permanecerán los cambios.

Un ejemplo de transacción sería una transferencia bancaria, donde es importantísimo que, si se descuenta la cantidad a transferir de una cuenta, esa cantidad sea sumada en la cuenta destino, en caso de que el sistema se caiga en la mitad del procedimiento, es necesario que la base de datos vuelva a un estado consistente.

Como se vio brevemente en el apartado de creación de tablas, MySQL soporta distintos tipos de tablas entre las que se destacamos MyISAM e InnoDB. Este último tipo, InnoDB, son las que permiten transacciones.

Los pasos para usar transacciones en MySQL son:

- Iniciar una transacción con el uso de una sentencia START TRANSACTION o BEGIN.
- Actualizar, insertar o eliminar registros en la base de datos.
- Si se quieren conservar los cambios realizados a la base de datos, completar la transacción con el uso de la sentencia COMMIT. Únicamente cuando se ejecuta este comando los cambios hechos serán permanentes.
- Si sucede algún problema, podemos hacer uso de la sentencia ROLLBACK. Esto me permite cancelar los cambios que han sido realizados, dejado la base de datos en el mismo estado que estaba cuando se inició la transacción.

Normalmente, cuando nos conectamos a MySQL, está por defecto activado el modo de ejecución automática (AUTOCOMMIT=ON). Esto significa que cada comnado SQL que se ejecture se considera una transacción distinta. Si, en el modo de ejecución automática activado, queremos realizar transacciones que engloben múltiples sentencias, hay que indicarlo explícitamente con los comandos START TRANSACTION o BEGIN, y terminar la transacción con COMMIT o ROLLBACK.

Podemos desactivar el modo de ejecución automática estableciendo AUTOCOMMIT=OFF. A partir de este momento, siempre tenemos una transacción empezada que finalizaraá en el momento que se ejecute una sentencia COMMIT o ROLLBACK.

Para comprobar si tenemos el modo de ejecución automátia activado o desactivado debemos ejecutar el comando:

```
SHOW VARIABLES LIKE 'AUTOCOMMIT';
```

Si deseamos activar o desactivar el modo de ejecución automática según corresponda, simplemente habrá que cambiar el valor de esta variables de la siguiente forma:

```
SET AUTOCOMMIT=ON;
O
SET AUTOCOMMIT=OFF;
```

Ejemplo: Ejecuta las siguientes instrucciones y observa cómo va cambiando el contenido de la tabla alumno.

```
/*Empiezo una transacción*/
BEGIN;

/*Inserto dos alumnos en la base de datos*/
INSERT INTO alumno (dni, nombre, apellido, email)
VALUES ('32653265P', 'Alicia', 'Romero', 'alicia@yahoo.es'),
                ('25142514L', 'Paco', 'Rivera', 'p.rivera@gmail.com');

/*Compruebo si se han añadido los dos alumnos mostrando el contenido de
la tabla alumno*/
SELECT * FROM alumno;

/*Vuelvo al estado al que estaba la base de datos justo antes de comenzar la
transacción*/
ROLLBACK;

/*Compruebo que se han deshecho los cambios que realicé sobre la base de
datos en esta transacción*/
SELECT * FROM alumno;
```

Ejemplo: Ejecuta las siguientes instrucciones y observa de nuevo cómo va cambiando el contenido de la tabla alumno.

```
/*Empiezo una transacción*/
BEGIN;

/*Inserto dos alumnos en la base de datos*/
INSERT INTO alumno (dni, nombre, apellido, email)
VALUES ('32653265P', 'Alicia', 'Romero', 'alicia@yahoo.es'),
                ('25142514L', 'Paco', 'Rivera', 'p.rivera@gmail.com');

/*Compruebo si se han añadido los dos alumnos mostrando el contenido de
la tabla alumno*/
SELECT * FROM alumno;

/*Finalizo la transacción haciendo persistentes los cambios realizados*/
COMMIT;
```

```
/*Compruebo que se mantienen los cambios que realicé sobre la base de
datos en esta transacción*/
SELECT * FROM alumno;
```

Hay comandos SQL que, dadas sus características, implican siempre confirmación automática. Estas son:

- Comandos que permiten definir o modificar objetos de la base de datos: CREATE, ALTER, DROP, RENAME, TRUNCATE.
- Comandos que permiten conceder o eliminar privilegios de acceso a las tablas de una base de datos: GRANT y REVOKE.
- Instrucciones de control de transacciones y bloqueo de tablas: START TRANSACTION, BEGIN, LOCK y UNLOCK.
- Instrucciones de carga de datos: LOAD DATA.

09. Políticas de bloqueo

Bloquear una tabla permite que nadie más pueda hacer uso de la misma, teniendo por tanto la exclusividad de lectura y/o escritura sobre ella. El bloqueo de tablas me va a permitir evitar los problemas ocasionados por el acceso a los mismos datos de dos transacciones distintas (por ejemplo, si una transacción lee datos escritos por otra transacción que no ha hecho COMMIT).

Existen dos tipos de bloqueo de tablas: de lectura y de escritura.

- El bloqueo de lectura va a permitir que se sigan realizando consultas sobre una tabla que está bloqueada, pero va a impedir que se realicen escrituras sobre la misma.
- El bloqueo de escritura va a impedir cualquier tipo de acceso sobre la tabla bloqueada.

El comando utilizado para bloquear tablas es LOCK TABLE y su sintaxis es:

```
LOCK TABLE nombre_tabla { READ | WRITE };
```

Para eliminar el bloque sobre una tabla se utiliza el comando UNLOCK TABLES. Hay que destacar que, aunque tengas bloqueos sobre varias tablas, una vez que se ejecuta este comando se desbloquean todas las tablas. Su sintaxis simplemente es:

```
UNLOCK TABLES;
```

Ejemplo: Abre dos ventanas de la consola MySQL y en ambas selecciona la base de datos centro de estudios como base de datos activa ejecutando el comando

```
USE centroEstudios;
```

En la primera ventana vamos a bloquear la tabla asignatura:

```
LOCK TABLE asignatura READ;
```

En la segunda ventana vamos a ejecutar una consulta y una inserción (vemos puedo realizar el primer comando, pero no el segundo):

```
SELECT * FROM asignatura;

INSERT INTO asignatura VALUES
    ('GVEC_MD', 'Marketing Digital', 'En esta asignatura...', 195,
    'GVEC', 1, NULL);
```

La segunda ventana permanecerá a la espera de que la primera ventana desbloquee la tabla:

```
UNLOCK TABLES;
```

010. Bibliografía

Hueso Ibañez, L. (2012). Bases de datos. Editorial Ra-Ma.

López Montalbán, I. y de Castro Vázquez, M. (2014). Gestión de bases de datos. Editorial Garceta.

Date, C. J. (2001). Introducción a los sistemas de bases de datos. Editorial Pearson Prentice Hall.

De Miguel, A.; Piattini, M.; Marcos, E. (1999). Diseño de bases de datos relacionales. Editorial Ra-Ma.

(2020) MySQL. SELECT Sintex. Recuperado de: https://dev.mysql.com/doc/refman/5.7/en/select.html

(2020) MySQL. Data Types. Recuperado de: https://dev.mysql.com/doc/refman/5.7/en/data-types.html

(2020) MySQL. CREATE TABLE Syntax. Recuperado de: https://dev.mysql.com/doc/refman/5.7/en/create-table.html

(2020) MySQL. INSERT Syntax. Recuperado de: http://dev.mysql.com/doc/refman/5.7/en/insert.html

(2020) MySQL. UPDATE Syntax. Recuperado de: http://dev.mysql.com/doc/refman/5.7/en/update.html

(2020) MySQL. DELETE Syntax. Recuperado de: http://dev.mysql.com/doc/refman/5.7/en/delete.html

(2020) MySQL. LOAD DATA INFILE Syntax. Recuperado de: http://dev.mysql.com/doc/refman/5.7/en/load-data.html

(2020) MySQL. ALTER TABLE Syntax. Recuperado de: http://dev.mysql.com/doc/refman/5.7/en/alter-table.html